

title: TDD — KZTEK Labeling Studio Cải Tiến Toàn Diện

plan: docs/plans/PLAN-labeling-studio-improve-2026-08-04/PLAN-MASTER.md

step: 0.2

author: tech-lead

reviewer: cto

created: 2026-08-04

updated: 2026-08-04

status: approved

TDD — KZTEK Labeling Studio Cải Tiến Toàn Diện

0. Bối cảnh

- Repo: `e:\KZTEK\Code_Git\Roboflow - Copy`, nhánh `Improve`.
- Nguồn kiến trúc: `code-graph/CODE-GRAPH.md` (STEP-0.1, commit `7118fe1`) — đây là nguồn sự thật về codebase hiện tại; tài liệu này KHÔNG lặp lại nội dung đó, chỉ tham chiếu.
- Phạm vi: quyết định kiến trúc cho 22 bước code còn lại (Phase 1–6 của PLAN-MASTER), tránh các bước sau tự suy đoán.
- Nguyên tắc chung: **giữ pattern hiện có** (better-sqlite3 raw SQL, Express modular router, React 18/TS thin API client) — chỉ thêm layer mới khi thật sự cần.

1. Goals / Non-goals

Goals:

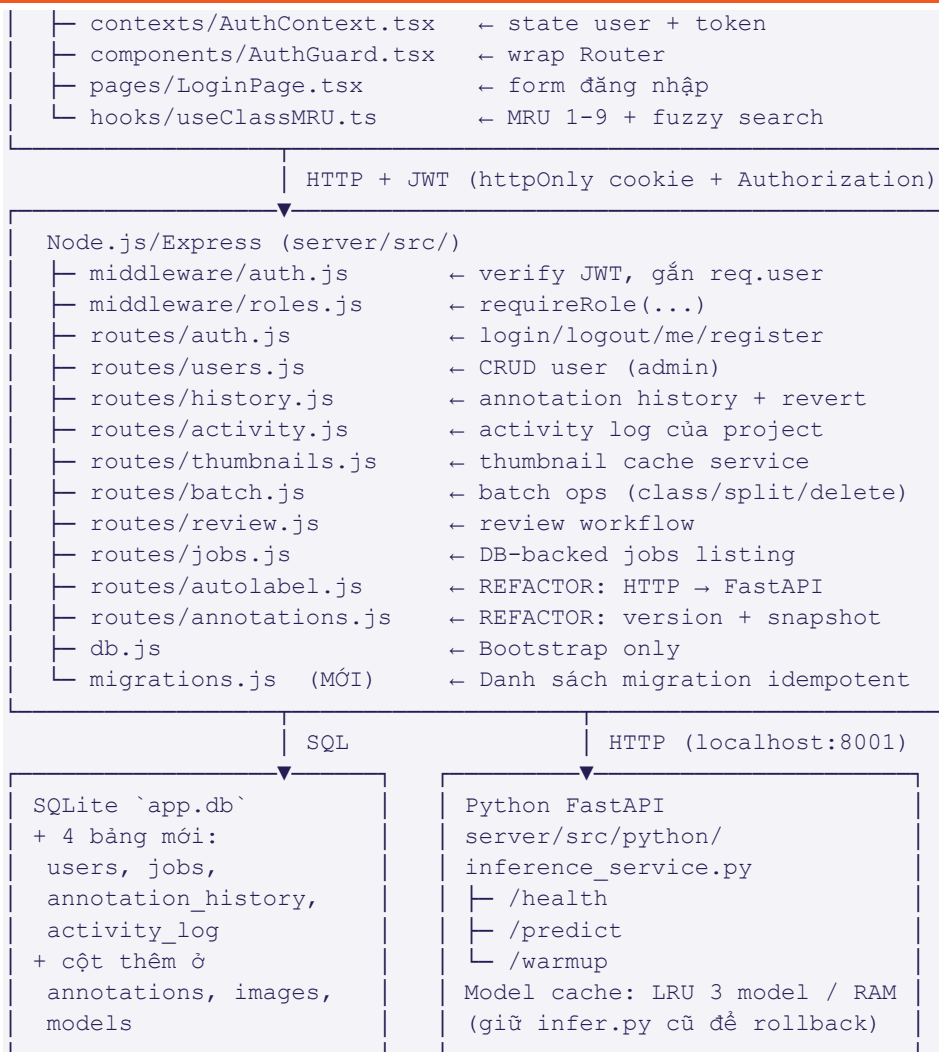
- Đóng chốt 5 điểm kiến trúc mấu chốt: (1) Inference service Python thường trực, (2) Auth model, (3) DB migration Phase 3, (4) Quick class switcher, (5) Image done status.
- Định nghĩa API contract mới cho toàn bộ tính năng thêm vào — client/api.ts và server routes có sẵn skeleton để tuân theo.
- Duy trì backward compat cho dữ liệu SQLite hiện có — không mất dữ liệu, không cần re-upload ảnh/model.

Non-goals:

- Không thay đổi stack (không đổi Express → NestJS, không đổi SQLite → Postgres, không đổi React → Next).
- Không viết code migration/ORM tool mới — giữ inline migration pattern trong `db.js` (nhưng tách file cho dễ đọc).
- Không thiết kế chi tiết UI mockup (thuộc bước 5.x, chỉ định API + state shape).

2. Kiến trúc tổng thể sau cải tiến

```
Browser (React 18, thêm AuthContext + AuthGuard)
client/src/
```



Nguyên tắc process lifecycle: Node `server/src/index.js` khi khởi động sẽ `spawn('python', ['-m', 'uvicorn', 'inference_service:app', '--port', '8001', '--host', '127.0.0.1'])` và giữ tham chiếu process; đăng ký `process.on('exit' | 'SIGINT' | 'SIGTERM')` để `kill()` con. **KHÔNG** dùng `systemd` — cùng lifecycle với Node, dev-friendly, không cần root; production Windows/Linux đều chạy tương đương.

3. Quyết định 1 — Inference service Python thường trực (Phase 1.1)

Điểm	Quyết định	Lý do
---	---	---
Framework	FastAPI + uvicorn	Async native, auto OpenAPI, type-safety với pydantic, cộng đồng lớn hơn Flask cho ML serving
Giao tiếp	HTTP loopback `127.0.0.1:8001`	Cross-platform (Windows dev + Linux prod), debug bằng curl dễ, không phụ thuộc Unix socket
Lifecycle	Node spawn 1 lần khi start, giữ sống, kill khi shutdown	Cùng vòng đời với server, không cần ops (systemd) cho triển khai single-node
Port	8001 (env <code>INFERENCE_PORT</code> , mặc định 8001)	Không đụng 4000 (Node) hay 5173 (Vite dev)
Health check	GET /health (Node poll mỗi 5s)	Đảm bảo service sẵn sàng trước

	trong 30s đầu, sau đó chỉ khi request lỗi)	khi nhận job
Model cache	Dict trong RAM, LRU cap 3 model (theo <code>model_path</code>)	Model YOLO ~50–200MB/model; cap 3 để tránh OOM trên máy 8GB
Fallback	Nếu health fail → autolabel trả 503 <code>'INFERENCE_UNAVAILABLE'</code> , KHÔNG fallback spawn <code>infer.py</code>	Tránh 2 code path song song gây nhầm lẫn; user thấy lỗi rõ, có thể restart server
Giữ infer.py cũ	CÓ — không xóa, chỉ ngừng gọi	Rollback nhanh nếu FastAPI có vấn đề production

3.1 API nội bộ (Python ↔ Node)

`GET /health`

```
{ "status": "ok", "loaded_models": ["/abs/path/a.pt", "/abs/path/b.pt"], "uptime_s": 1234 }
```

`POST /warmup` (optional, gọi khi user chọn model trong AutoLabelModal)

```
Request: { "model_path": "/abs/path/model.pt" }
Response: { "loaded": true, "model_path": "...", "took_ms": 1523 }
```

`POST /predict` (thay thế toàn bộ giao thức stdin/stdout của `infer.py`)

```
Request:
{
  "model_path": "/abs/path/model.pt",
  "conf": 0.25,
  "images": [{ "id": "nanoid", "path": "/abs/path/img.jpg" }, ...]
}

Response (đồng bộ, 1 lần trả toàn bộ – không streaming vì Node giờ có bảng `jobs` để track progress qua polling):
{
  "results": [
    {
      "image_id": "nanoid",
      "detections": [
        { "class_index": 0, "type": "bbox", "x": 12, "y": 20, "w": 50, "h": 80 },
        { "class_index": 1, "type": "quad", "points": [{"x":1,"y":2}, ...] }
      ]
    },
    ...
  ],
  "errors": [{ "image_id": "nanoid", "error": "message" }]
}
```

Batch size: Node chia mỗi request tối đa 32 ảnh; job lớn hơn được chia nhiều request tuần tự, cập nhật `jobs.done` sau mỗi batch (progress rõ ràng cho UI).

3.2 Contract Node ↔ Client (không đổi)

`POST /api/projects/:pid/auto-label` và `GET /api/projects/:pid/auto-label/:jid` giữ nguyên response shape — client không cần đổi.

4. Quyết định 2 — Auth (Phase 2)

4.1 Token scheme

Chọn: JWT trong httpOnly cookie + Authorization header (dual-mode).

Điểm	Quyết định	Lý do
---	---	---
Scheme	JWT (HS256)	Stateless, không cần bảng sessions, phù hợp SPA đơn node
Lib server	jsonwebtoken ^9.x	Chuẩn de-facto Node, bảo trì tốt
Lib hash	bcrypt ^5.x, cost=12	Chuẩn công nghiệp; scrypt/argon2 dư thừa cho scope này
Vị trí lưu	httpOnly cookie kztek_token (browser) + Authorization: Bearer fallback (Postman/CI)	XSS-resistant khi web, dễ dev khi test API
TTL	24h, không refresh token (Phase 2)	Đơn giản; thêm refresh khi có yêu cầu thực
Secret	env AUTH_JWT_SECRET (bắt buộc, server refuse start nếu thiếu)	Không hardcode, không có default fallback
CSRF	SameSite=Lax + double-submit token cho POST/PUT/PATCH/DELETE	Balance an toàn/UX
Rate limit	Login: 10 lần / IP / 15 phút (in-memory bucket)	Chống brute force, đủ cho single-node

4.2 Bảng `users`

```
CREATE TABLE IF NOT EXISTS users (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  username    TEXT      NOT NULL UNIQUE,
  password_hash TEXT    NOT NULL,
  display_name TEXT    NOT NULL,
  role        TEXT      NOT NULL CHECK(role IN ('annotator','reviewer','admin')),
  color       TEXT      NOT NULL DEFAULT '#4A3F8C', -- KZTEK sub-heading, mặc định
  is_active   INTEGER   NOT NULL DEFAULT 1,         -- 0 = disabled
  created_at  TEXT      NOT NULL DEFAULT (datetime('now')),
  last_login_at TEXT
);
CREATE UNIQUE INDEX IF NOT EXISTS idx_users_username ON users(username);
```

Ghi chú kiểu id: `users.id INTEGER AUTOINCREMENT` — khác với các bảng cũ (dùng TEXT nanoid) nhưng chấp nhận được vì (a) users là domain mới, (b) FK từ `annotation_history.actor_id`, `activity_log.actor_id`, `images.completed_by` sẽ đồng nhất INTEGER — đơn giản hơn TEXT.

Roles cụ thể:

Role	Quyền
---	---
annotator	Đọc mọi project; ghi annotations/images (upload, edit, delete ảnh mình upload); mark done
reviewer	Toàn quyền của annotator + revert history của người khác + approve/reject review + xóa ảnh bất kỳ
admin	Toàn quyền + quản lý users + xóa project + đổi role

Enforcement: `middleware/roles.js` — decorator `requireRole('reviewer', 'admin')` gắn vào từng route.

4.3 Middleware layout

- **Global:** `middleware/auth.js` — parse cookie/header, verify JWT, gắn `req.user = { id, username, role, color }`. Áp cho toàn bộ `/api/*` trừ `/api/auth/login` và `/api/auth/refresh` (nếu có sau này). 401 nếu invalid.
- **Per-route:** `requireRole()` chỉ áp cho endpoint ghi (POST/PUT/PATCH/DELETE) hoặc endpoint admin. Đọc (GET) chỉ cần authenticated.

4.4 API contract

Method	Path	Body	Response	Role
---	---	---	---	---
POST	<code>/api/auth/login</code>	<code>{username, password}</code>	<code>{user, token}</code> + Set-Cookie	
POST	<code>/api/auth/logout</code>	—	204 + Clear-Cookie	authenticated
GET	<code>/api/auth/me</code>	—	<code>{user}</code>	authenticated
POST	<code>/api/auth/register</code>	<code>{username, password, display_name, role, color?}</code>	<code>{user}</code> (201)	admin
GET	<code>/api/users</code>	—	<code>User[]</code>	admin
PATCH	<code>/api/users/:id</code>	<code>{display_name?, role?, color?, is_active?, password?}</code>	<code>{user}</code>	admin (self cho display_name/color/password)
DELETE	<code>/api/users/:id</code>	—	204	admin

Seed admin đầu tiên: khi migration tạo bảng `users` và bảng đang trống → tự tạo admin từ env `AUTH_BOOTSTRAP_ADMIN_USER/AUTH_BOOTSTRAP_ADMIN_PASSWORD`; nếu env thiếu → tạo `admin/kztek@2026` và log warning yêu cầu đổi ngay.

5. Quyết định 3 — DB migration Phase 3

5.1 Cách chạy migration

Giữ pattern hiện có (idempotent PRAGMA + ALTER TABLE) nhưng tách sang file ``server/src/migrations.js`` — dễ đọc, dễ audit, không đổi runtime.

- `db.js` chỉ giữ CREATE TABLE (initial schema).
- `migrations.js` export `runMigrations(db)` — chạy lần lượt các function `m001_add_annotation_type_points()`, `m002_add_classes_hotkey()` (đã có, chuyển từ `db.js` sang), `m003_add_users()`, `m004_add_annotation_versioning()`, `m005_add_annotation_history()`, `m006_add_activity_log()`, `m007_add_image_completed()`, `m008_add_jobs()`, `m009_add_detect_cache()`, `m010_add_model_metadata()`.

- Mỗi function tự check tồn tại (`PRAGMA table_info/SELECT name FROM sqlite_master WHERE name=?`), an toàn khi chạy lại.
- `index.js` gọi `runMigrations(db)` ngay sau khi mở DB (trước khi mount route).

Backup DB: trước khi chạy migration đầu tiên trong session mà detect có migration mới sẽ chạy → copy `app.db` → `app.db.bak-{YYYYMMDD-HH:mm:ss}`. Giữ 5 file backup gần nhất, xóa cũ hơn.

5.2 Schema thêm — chi tiết SQL

```
-- m004: Thêm cột version + updated_at + updated_by vào annotations
ALTER TABLE annotations ADD COLUMN version      INTEGER NOT NULL DEFAULT 1;
ALTER TABLE annotations ADD COLUMN updated_at TEXT;                                -- ISO, cập
nhật mỗi lần PUT
ALTER TABLE annotations ADD COLUMN updated_by INTEGER;                                -- FK users(id),
SQLite không enforce FK khi ALTER

-- m005: Bảng snapshot lịch sử toàn bộ annotation của 1 ảnh tại mỗi lần save
CREATE TABLE IF NOT EXISTS annotation_history (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  image_id    TEXT      NOT NULL REFERENCES images(id) ON DELETE CASCADE,
  version     INTEGER NOT NULL,                -- version của ảnh sau lần save này
  snapshot    TEXT      NOT NULL,              -- JSON: [{id,class_id,x,y,w,h,type,points},...]
  actor_id    INTEGER NOT NULL REFERENCES users(id),
  action      TEXT      NOT NULL CHECK(action IN ('create','update','delete','revert')),
  created_at  TEXT      NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_ah_image_version ON annotation_history(image_id, version
DESC);

-- m006: Bảng log hành động cấp project
CREATE TABLE IF NOT EXISTS activity_log (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  project_id  TEXT      NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  actor_id    INTEGER NOT NULL REFERENCES users(id),
  action      TEXT      NOT NULL,              --
'upload'|'export'|'split_change'|'delete_image'|'model_upload'|'model_delete'|'class_ch
ange'
  entity_type TEXT,                            -- 'image'|'model'|'class'|'project'
  entity_id   TEXT,                            -- id của entity liên quan (nullable)
  metadata    TEXT,                            -- JSON free-form: {count, format, from_split,
to_split, ...}
  created_at  TEXT      NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_activity_project_time ON activity_log(project_id,
created_at DESC);

-- m007: Đánh dấu ảnh "Xong"
ALTER TABLE images ADD COLUMN completed_at TEXT;                                -- NULL = chưa
done
ALTER TABLE images ADD COLUMN completed_by INTEGER;                                -- FK users(id),
nullable

-- m008: Persist jobs (Phase 1.2)
CREATE TABLE IF NOT EXISTS jobs (
  id          TEXT      PRIMARY KEY,          -- nanoid (giữ tương thích client hiện có)
  project_id  TEXT      NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
```

```

kind      TEXT      NOT NULL,          -- 'auto_label' | (mở rộng sau)
status    TEXT      NOT NULL,          --
'pending'|'running'|'done'|'failed'|'cancelled'
total     INTEGER NOT NULL DEFAULT 0,
done      INTEGER NOT NULL DEFAULT 0,
created   INTEGER NOT NULL DEFAULT 0,
failed    INTEGER NOT NULL DEFAULT 0,
error     TEXT,
payload   TEXT,                        -- JSON: {model_id, conf, image_ids[],
unmatchedClasses[]}
actor_id   INTEGER REFERENCES users(id),
created_at TEXT      NOT NULL DEFAULT (datetime('now')),
updated_at TEXT      NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_jobs_project_time ON jobs(project_id, created_at DESC);

-- m009: Detect cache (Phase 4.1)
CREATE TABLE IF NOT EXISTS detect_cache (
  image_id   TEXT      NOT NULL REFERENCES images(id) ON DELETE CASCADE,
  model_id   TEXT      NOT NULL REFERENCES models(id) ON DELETE CASCADE,
  conf       REAL      NOT NULL,
  detections TEXT      NOT NULL,        -- JSON payload từ inference_service
  created_at TEXT      NOT NULL DEFAULT (datetime('now')),
  PRIMARY KEY(image_id, model_id, conf)
);

-- m010: Model metadata (Phase 6.1)
ALTER TABLE models ADD COLUMN metadata TEXT;                                -- JSON:
{trained_at, dataset_id, map50, map95, notes}

```

5.3 Snapshot vs Diff cho `annotation\_history`

Quyết định: SNAPSHOT toàn bộ annotations của ảnh sau mỗi save.

Lý do: Mỗi ảnh trung bình 5-20 box, snapshot ~1-5 KB. Với 100K save/năm ~200-500 MB — chấp nhận được. Snapshot đơn giản, revert 1 lệnh; diff phải tự viết apply logic. Nếu tương lai kích thước vấn đề → thêm compress + retention (giữ history 90 ngày).

6. Quyết định 4 — Quick class switcher (5.5)

Xác nhận approach từ PLAN-MASTER + chốt chi tiết:

Điểm	Quyết định
---	---
MRU 1-9	Bấm số 1-9 chọn class dùng gần nhất theo thứ tự MRU của user hiện tại trong project hiện tại
Storage MRU	localStorage per-user per-project — key <code>kztek.mru.{userId}.{projectId}</code> = <code>string[]</code> (classId theo thứ tự MRU, max 9)
Fuzzy search	Ctrl+K mở modal, input filter theo subsequence match trên class name (không cần lib mới, tự viết ~30 dòng)
Hotkey 2 ký tự	Mở rộng <code>classes.hotkey</code> từ 1 → tối đa 2 ký tự. Sửa <code>normalizeHotkey</code> trong <code>classes.js</code> cho phép 1-2 ký tự
Backend đổi	KHÔNG — không cần table mới, chỉ đổi validation hotkey

UI	AnnotatorPage: overlay bảng hotkey (toggle bằng ?); modal Ctrl+K global
----	--

Không dùng DB cho MRU vì (1) preference cá nhân, (2) mất khi clear browser không nghiêm trọng, (3) tránh N+1 query khi mở AnnotatorPage.

7. Quyết định 5 — Image done status (3.5)

Xác nhận: 2 field độc lập, KHÔNG đổi enum `status`.

Field	Ý nghĩa
---	---
<code>images.status (unlabeled/labeled)</code>	Backend tự set dựa trên có annotation hay không. GIỮ NGUYỄN.
<code>images.completed_at (NULL/ISO)</code>	User action — "Đã xong, không sửa nữa"
<code>images.completed_by (NULL/user.id)</code>	Ai đánh dấu xong

- Cho phép `labeled + completed_at IS NULL` (đã có box, chưa xác nhận xong) → chờ QA/reviewer.
- Cho phép `unlabeled + completed_at NOT NULL` (ảnh xác nhận không có object nào — negative sample).
- Endpoint: reuse `PATCH /api/projects/:pid/images/:iid` với body mở rộng: `{completed: true/false}` — server set `completed_at = datetime('now'), completed_by = req.user.id` (hoặc NULL khi undo).
- UI hotkey đề xuất: `Ctrl+Enter` trong AnnotatorPage (bước 3.5 sẽ chốt).

8. API contracts mới — tổng hợp

Ngoài Auth (§4.4), các endpoint mới:

8.1 History (Phase 3.2)

Method	Path	Response	Role
---	---	---	---
GET	<code>/api/images/:iid/history</code>	<code>HistoryEntry[]</code> (mới nhất trước)	authenticated
GET	<code>/api/images/:iid/history/:version</code>	<code>{version, snapshot, actor, created_at}</code>	authenticated
POST	<code>/api/images/:iid/history/:version/revert</code>	<code>Annotation[]</code> (mới sau revert)	reviewer, admin

8.2 Activity log (Phase 3.3)

Method	Path	Response
---	---	---
GET	<code>/api/projects/:pid/activity? page=&limit=&action=</code>	<code>{items: ActivityEntry[], total, page}</code>

8.3 Jobs persist (Phase 1.2)

Method	Path	Response
---	---	---
GET	/api/projects/:pid/jobs?kind=&status=	Job[] (mới nhất trước)
POST	/api/projects/:pid/jobs/:jid/cancel	Job

8.4 Thumbnails (Phase 1.3)

Method	Path	Response
---	---	---
GET	/api/thumbnails/:imageId?size=200	image/jpeg (cache 7 ngày, ETag)

- Backend: thư mục cache `server/data/thumbnails/<projectId>/<imageId>-<size>.jpg`, sinh lazy khi request đầu.

8.5 Batch ops (Phase 5.3)

Method	Path	Body	Response
---	---	---	---
POST	/api/projects/:pid/images/batch	{imageIds: string[], action: 'set_class'}	{set_split\ 'delete', payload}\ {affected: n}

8.6 Review workflow (Phase 2.3)

Method	Path	Body	Role
---	---	---	---
POST	/api/images/:iid/review	{decision: 'approve'}	'reject', note?} reviewer, admin

Trạng thái review lưu qua `completed_at` (approve = set completed) + `activity_log.action='review_reject'` (không cần cột mới trong `images`).

9. Rủi ro & giảm thiểu

Rủi ro	Mức	Giảm thiểu
---	---	---
Migration bị chạy nửa chừng → schema hỏng	Cao	Backup <code>.db.bak-{ts}</code> trước, mỗi function migration đóng gói trong <code>db.transaction(...)</code> , log rõ ràng
FastAPI process chết giữa job → jobs treo <code>running</code>	Trung bình	Node health-check 30s/lần; nếu unhealthy → cập nhật <code>jobs.status='failed'</code> cho job <code>running</code> cũ hơn 5 phút mà không có heartbeat
JWT secret leak / brute force	Cao	Bắt buộc <code>AUTH_JWT_SECRET</code> env

		≥ 32 ký tự, rate-limit login, log audit vào <code>activity_log</code>
Bcrypt cost=12 quá chậm trên máy yếu	Thấp	Đo thực tế trong Phase 2.2; giảm xuống 10 nếu > 200ms/hash
Snapshot annotation_history phình DB	Trung bình	Giới hạn giữ 200 version/ảnh gần nhất, xoá cũ hơn bằng cron nhẹ (setInterval 1h)
FK từ INTEGER users.id sang bảng có TEXT id không nhất quán	Thấp	Chỉ FK 1 chiều users.id ← INTEGER; các bảng cũ TEXT id vẫn dùng nanoid
Class hotkey 2 ký tự đụng phím tắt trình duyệt	Trung bình	Chỉ chấp nhận [A-Za-z0-9] + shift, chặn Ctrl/Cmd; validate ở <code>normalizeHotkey</code>
Batch delete nhầm cả ngàn ảnh	Cao	Endpoint batch yêu cầu <code>confirm: true</code> trong body + <code>activity_log</code> entry; UI double-confirm

10. Task breakdown (map sang PLAN-MASTER)

Không đổi 22 bước trong PLAN-MASTER — TDD này CUNG CẤP quyết định cho các bước đó. Mọi bước code (1.1→6.3) tuân theo API contract + schema trong tài liệu này; nếu có thay đổi thiết kế trong quá trình implement → PHẢI update TDD này + bump `updated` ở frontmatter.

11. Test strategy tóm tắt

- Auth: unit test bcrypt hash/verify, JWT sign/verify, middleware 401/403; integration test login → protected route.
- Migration: chạy trên bản `app.db` clone thực tế, verify không mất dữ liệu, `annotations` vẫn đọc được.
- Inference service: contract test `/predict` với 3 model mẫu; test load 100 ảnh xem có OOM không.
- History revert: golden test — save v1 → v2 → revert v1 → verify byte-equal snapshot.

12. Rollback plan

Nếu ...	Rollback
---	---
FastAPI service hỏng	Env <code>USE_LEGACY_INFER=1</code> → autolabel spawn <code>infer.py</code> cũ (giữ code)
Migration Phase 3 hỏng	Dừng server, restore <code>app.db.bak-{ts}</code> gần nhất, kiểm tra
Auth phá client production	Tạm env <code>AUTH_DISABLED=1</code> (middleware inject <code>req.user=admin bootstrap</code>) — CHỈ cho hotfix, đóng lại sau khi fix xong

13. Tham chiếu

- ADR: `docs/architecture/ADR-labeling-studio-improve.md`
- Plan: `docs/plans/PLAN-labeling-studio-improve-2026-08-04/PLAN-MASTER.md`
- CODE-GRAPH: `code-graph/CODE-GRAPH.md` (commit 7118fe1)